Pedal Pro Rentals

Agentforce Project — Study & Revision Guide

This is a study document, not just a build log. It's organized so you can re-explain this project from memory — in an interview, a viva, or to yourself six months from now — without needing to reopen Salesforce. Each major section ends with a short self-test (Q&A) so you can check you've actually internalized it, not just read it.

Table of Contents

- 1. The 30-Second Pitch (memorize this)
- 2. Architecture Mental Model
- 3. Data Model — Objects and Fields
- 4. The 5 Flows — What Each One Does and Why
- 5. The Agent — Subagents, Routing, Actions
- 6. The Guardrail — Deep Dive
- 7. Full Bug Log — Chronological, With the Underlying Principle Each One Teaches
- 8. Permissions & Sharing — The Two Different Problems
- 9. Self-Test — Could You Explain This Cold?
- 10. Glossary of Terms Used
- 11. Quick Reference Card (print this one page)

1. The 30-Second Pitch

Say this, or something close to it, when asked "tell me about this project":

```
"I built a custom Agentforce agent for a fictional bike rental company. It handles three things: checking live bike availability, booking a rental end-to-end through a Salesforce Flow that creates a record and updates inventory status, and cancelling a booking — but with a business guardrail: if the rental starts within 2 hours, the agent refuses to cancel it and instead escalates to a human by creating a Case. The interesting part was the debugging — things that worked perfectly when I tested a Flow manually kept failing silently when the live agent called it, which taught me a lot about the difference between the admin user's context and the agent's own dedicated user and its permissions."
```

Note: That last sentence is your strongest material. Anyone technical will immediately respect "things worked in Debug but not live, and I tracked down why" — it signals real debugging skill, not tutorial-following.

2. Architecture Mental Model

Picture it in three layers, top to bottom:

Layer 1 — The Agent (Agentforce Studio)

A router at the top ("Agent Router") listens to what the customer says and hands the conversation to one of three **subagents**: bike availability, bike booking, booking cancellation. Each subagent has its own **Reasoning Instructions** (plain-English rules telling the LLM what to do and when) and a list of **Actions** it's allowed to call.

Layer 2 — Actions

An Action is Agentforce's wrapper around something that actually does work. In this project, every single Action is type **Flow** — meaning when the agent decides to call an action, it's really invoking a Salesforce Flow behind the scenes, passing in whatever inputs the LLM has gathered from the conversation.

Layer 3 — Flows (the real logic)

Five Flows do all the actual work: querying Bike__c, creating Rental__c records, calculating time differences, updating statuses, and creating Case records. **This is the layer where real business logic and precise math live** — the LLM layer above is for language and judgment calls, not arithmetic.

Why this layering matters

The single biggest lesson of this whole build: **the LLM is unreliable at precise things** (exact dates, exact minute counts, exact record IDs) but **very good at fuzzy things** (understanding "e-bike" means "Electric", deciding which subagent to route to, writing a natural confirmation message). Every real bug in this project came from asking the LLM layer to do something that belonged in the Flow layer instead.

Q: Why not just let the agent's reasoning do the 2-hour cancellation check directly?

A: Because the LLM referencing a variable like "minutes_until_start" that nothing ever calculated is exactly the kind of precise-math task it's bad at. The fix was to calculate it as a real Number in a Flow Formula resource and hand the agent a ready-made answer to branch on.

3. Data Model

Bike__c

Field	API Name	Type
Model	Model__c	Text(80)
Type	Type__c	Picklist: Mountain, Road, Hybrid, Electric
Location	Location__c	Text(120)
Status	Status__c	Picklist: Available, Rented, Maintenance
Hourly Rate	Hourly_Rate__c	Currency(6,2)

Rental__c

Field	API Name	Type
Bike	Bike__c	Lookup(Bike)
Customer	Customer__c	Lookup(Contact) — must be the standard Contact object
Start Date Time	Start_Date_Time__c	Date/Time
End Date Time	End_Date_Time__c	Date/Time
Status	Status__c	Picklist: Booked, Active, Completed, Cancelled
Late Fee	Late_Fee__c	Currency(6,2)

Q: Why does it matter that Customer__c points to the standard Contact object specifically?

A: Two reasons: first, an early mistake accidentally linked it to a different custom object, which silently broke lookups. Second — and more subtly — because it's Contact, record-level sharing and Org-Wide Defaults for Contact/Account govern whether the agent's dedicated user can actually read or write a given customer's rental, which became a real bug later (see Section 8).

4. The 5 Flows — What Each Does and Why

1. Book_Rental_Flow

Job: book a rental and mark the bike Rented, in one transaction.

Inputs: bikeId, customerId, startDateTimeText, endDateTimeText — all Text.

Logic: Get the Bike where Id=bikeId AND Status='Available' → Decision: found? → if yes: convert the two text dates to real Date/Time using Formula resources with DATETIMEVALUE(), Create the Rental__c, Update the Bike's Status to Rented, output success=true and a real confirmationId. If not found: output a "not available" message.

Note: Why Text inputs, converted inside the Flow, instead of Date/Time inputs directly? Agentforce could not reliably pass a real Date/Time value into the Flow when calling it live — it arrived as a generic "Object" type and the Flow failed silently. Taking Text and converting internally with DATETIMEVALUE() sidesteps that entirely.

2. Check_Bike_Availability

Job: tell the customer whether a type+location combination has an available bike.

Inputs: bikeType, location — Text.

Logic: Get all Bikes matching Type, Location, Status=Available → Decision: any found? → output a plain-text "available" or "not available" message.

Note: An earlier version tried to Loop over the results and build a formatted list of bike names. It worked in manual Debug but threw an UNKNOWN_EXCEPTION every time the live agent called it. Simplifying to a plain status message (no Loop) fixed it completely — a good example of trading a nice-to-have feature for reliability under a deadline.

3. Get_Booking_Name

Job: look up a booking and calculate how many minutes remain until it starts — this number powers the guardrail.

Inputs: bookingId — Text.

Logic: Get the Rental by Id → Assignment: copy out bookingStartTime, bookingStatus, bookingBikeId → a Formula resource `(Get_Booking.Start_Date_Time__c - NOW()) * 1440` calculates minutesUntilStart → Assignment copies that into the output variable minutesUntilStartOutput.

Watch out: This Flow's name ended up being "Get_Booking_Name", not "Get_Booking_Details" — a naming slip early on that was never worth correcting later since the action already referenced it correctly.

4. Cancel_Booking

Job: mark a rental Cancelled. Only ever called when the guardrail check says it's safe (more than 2 hours out).

Inputs: bookingId — Text. **Logic:** Update the Rental's Status to Cancelled, output success=true.

5. Escalate_To_Human

Job: the guardrail's safety valve. Creates a Case instead of cancelling.

Inputs: bookingId, Reason — Text. **Logic:** Create a Case (Subject: "Urgent rental change request", Description = Reason) → output caseId = {!Create_Escalation_Case.Id} specifically (not the whole record).

5. The Agent — Subagents, Routing, Actions

Subagent	Routes on	Calls
bike availability	"available", "find a bike", "what bikes"	Check Bike Availability v3
bike booking	"book", "rent", "reserve"	Book Rental v5
booking cancellation	"cancel", "cancellation"	Get Booking Details v4, Cancel Booking v2, Escalate To Human

Why the action version numbers (v2, v3, v4, v5...)

Agentforce did not reliably refresh an action's Inputs/Outputs when its underlying Flow's variables changed — editing in place left stale, mismatched fields behind. The reliable pattern that emerged: **delete the action, recreate it fresh via "Create Custom Action" pointed at the Flow**. Each recreation got a new version suffix to keep track of which one was actually current and attached.

Q: If an action's Inputs panel shows a Flow variable as type "Object" instead of something sensible like Text or Date, what does that usually mean in this project's experience?

A: That input type can't be reliably populated by the live agent call, even though manual Debug tolerates it. The fix used throughout this project was to change the Flow's input to Text and convert it internally with a Formula, rather than fight the type directly in the action config (which wasn't editable anyway).

Q: Why recreate actions instead of editing them?

A: Editing in place left outputs stuck as "Not assigned" or inputs stuck with stale names, even after the Flow itself was fixed and reactivated. Deleting and recreating via "Create Custom Action" → select the Flow fresh reliably pulled the real, current variable list every time.

6. The Guardrail — Deep Dive

Business rule: a booking cannot be cancelled by the AI if it starts within 2 hours. Must escalate to a human instead.

The journey to get this right

- **First attempt:** instructions referenced a variable "minutes_until_start" that nothing ever calculated. Result: every cancellation succeeded, guardrail never fired. Lesson — a variable name appearing in an instruction doesn't make it exist.
- **Fix:** added a Formula resource to Get_Booking_Name that genuinely computes the minutes difference, exposed it as minutesUntilStartOutput, and updated instructions to reference that exact name.
- **Second snag:** the newly-added Assignment element referencing the Formula silently didn't persist on the canvas the first time — always re-open and visually confirm new elements are really there after saving.
- **Timezone surprise:** the org's internal NOW() didn't match wall-clock expectations by roughly 12+ hours. Rather than chase the exact Setup timezone setting under time pressure, testing was done empirically — create a test record, run Debug immediately, read the real number, adjust the Start time until it landed in the desired range.
- **Final verification:** a rental 88-104 minutes out correctly triggered escalation and created a real Case (#00001028, #...); a rental ~814 minutes out correctly proceeded to cancellation.

The exact policy logic (as written in the subagent's instructions)

```
If minutesUntilStartOutput > 120: call Cancel Booking v2, confirm cancellation.
If minutesUntilStartOutput <= 120: call Escalate To Human v2, tell the customer a team member
will review it.
Never cancel a booking starting within 2 hours — always escalate those cases.
```

Q: Why 1440 in the Formula (Start_Date_Time__c - NOW()) * 1440?

A: Subtracting two DateTimes in Salesforce Flow gives you a difference measured in days (as a decimal). Multiplying by 1440 (the number of minutes in a day) converts that day-fraction into minutes, matching the 2-hour (120-minute) threshold used in the policy.

7. Full Bug Log — With the Principle Each One Teaches

#	Bug	Principle it teaches
1	Picklist field saved as Multi-Select with a wrong API name	Double-check radio buttons and the Field Name box before saving — these c
2	Customer__c lookup pointed to the wrong object	Always verify a Lookup field's "Related To" target explicitly, don't assume the
3	Successful bookings also got overwritten with "not available"	In Flow Builder, visually trace every branch to its own independent End — a b
4	"X is not defined in actions" errors	A tool_ prefix or any name mismatch between an instruction's text and the ac
5	Action input names didn't match Flow variable names	AI-drafted actions can guess wrong names before the real Flow exists — alwa
6	Action output stuck at "Not assigned"	A Get Records result isn't automatically an output — it must be explicitly assig
7	Date/Time inputs typed as "Object", failed silently live	Prefer Text inputs converted internally via Formula over trusting Agentforce to
8	Agent claimed success with no record actually created	Always verify against the real data, not the agent's stated response — and wr
9	Guardrail always cancelled, never escalated	A variable referenced in instructions must actually be calculated and output by
10	A new Flow element vanished after saving	Re-open and visually confirm new elements persisted — don't trust that a sav
11	Escalation claimed success, Case list showed nothing	List views can be filtered/owned differently than expected — verify a specific r
12	Agent's user had no access to custom objects at all	A dedicated automation/agent user is a separate identity from the admin build
13	"e-bike" sent instead of the picklist value "Electric"	An LLM will paraphrase user language unless explicitly told the exact allowed
14	Generic failure message masked the real error	Add a Fault Path with {!$Flow.FaultMessage} during debugging to see Salesf
15	"Insufficient access rights" on a specific Contact	Object-level permission (Permission Set) is not the same as record-level acce

8. Permissions & Sharing — Two Different Problems

This project hit both halves of Salesforce's access model, and it's worth being able to explain the difference clearly:

Object/Field-level permission (Permission Set)

Controls *what kinds of things* a user can do — can this user Read/Create/Edit ANY Bike__c record at all? This was fixed once, early on, with a custom Permission Set ("Agentforce Bike Access") granting the agent's dedicated user CRUD on Bike__c, Rental__c, and Case.

Record-level access (sharing / ownership / Org-Wide Defaults)

Controls *which specific records* a user can see or touch, even if they have full object permission. This surfaced much later: the agent's user had full Create/Edit rights on Rental__c in general, but still failed to book against one specific Contact record it wasn't the owner of and wasn't shared with, because that Contact's Account-level sharing was effectively private to the human admin.

The two fixes, side by side

Problem	Fix applied
Agent's user couldn't touch Bike__c/Rental__c/Case at all	Permission Set: Agentforce Bike Access, granting Read/Create/Edit on all tl
Agent's user couldn't book against one specific Contact	Changed that Contact record's Owner to the EinsteinServiceAgent User dire

Q: A booking works for one Contact but fails for another with the exact same action/Flow config — what's the likely category of problem?

A: Record-level sharing, not object permission — since object permission would block or allow ALL records uniformly. Check the specific record's Owner and any sharing rules.

9. Self-Test — Could You Explain This Cold?

Cover the rest of this document and answer these out loud. If you hesitate on one, that's exactly where to re-study.

- What are the 3 subagents and what does each one route on?
- Why does Book_Rental_Flow take dates as Text instead of Date/Time?
- What does the guardrail actually check, and where is that number calculated?
- Name 2 bugs where the Flow worked in Debug but failed when the agent called it live. What did they have in common?
- What's the difference between the Permission Set fix and the record-ownership fix, and which bug needed which?
- Why was the Loop removed from Check_Bike_Availability?
- What does {!$Flow.FaultMessage} do and why was it useful?
- Walk through what happens, step by step, when a customer asks to cancel a booking starting in 90 minutes.

10. Glossary

Term	Meaning in this project
Subagent	A focused unit inside the agent with its own instructions and allowed actions; the router hands conve
Action (Agentforce)	A callable capability the agent can invoke; here, every action wraps a Flow
Autolaunched Flow	A Flow with no screen/trigger, designed to be invoked programmatically — exactly what an Agentfor
Formula resource	A Flow resource that computes a value from an expression, e.g. a date-math calculation, without ne
Fault Path	An alternate branch in a Flow that runs when an element throws a runtime error, instead of the Flow
$Flow.FaultMessage	A system variable available on a Fault Path that holds Salesforce's real technical error text
Org-Wide Default (OWD)	The baseline sharing setting for an object, org-wide, before any more-open sharing rules are layered
Permission Set	A bundle of object/field-level permissions that can be assigned to any user, including a dedicated au
EinsteinServiceAgent User	The dedicated Salesforce user identity this agent runs as — separate from the human admin who b

11. Quick Reference Card

Flow	Inputs	Key output
Book_Rental_Flow	bikeId, customerId, startDateTimeText, endDateTimeText	success, confirmationId
Check_Bike_Availability	bikeType, location	resultMessage
Get_Booking_Name	bookingId	minutesUntilStartOutput
Cancel_Booking	bookingId	success
Escalate_To_Human	bookingId, Reason	caseId

Action (live)	Subagent
Check Bike Availability v3	bike availability
Book Rental v5	bike booking
Get Booking Details v4, Cancel Booking v2, Escalate To Human v2	booking cancellation

Guardrail threshold: 120 minutes. **Formula:** (Start_Date_Time__c - NOW()) * 1440

Permission Set: Agentforce Bike Access → Bike__c, Rental__c, Case → Read/Create/Edit

Agent user: EinsteinServiceAgent User (Profile: Einstein Agent User)

Study tip: don't just reread this top to bottom. Close it, try the Self-Test in Section 9 from memory, then open this back up only to check the parts you got stuck on.